

Introdução à Lógica e à Programação

Listas: parte I

Listas

Uma lista é um tipo de objeto capaz de armazenar múltiplos valores. Cada valor armazenado pela lista é chamado de **elemento**, os quais podem ser de qualquer tipo. O **tamanho** de uma lista é igual à sua quantidade de elementos, o qual pode ser obtido por meio da função `len`.

```
a = [1, 2, 3, 4]
b = ['A', 5, 3.8]
c = []
print(len(a)) # 4
print(len(b)) # 3
print(len(c)) # 0
```

Listas

Assim como as strings, a instrução `for` pode ser usada para iterar os elementos de uma lista.

```
a = [1, 2, 3, 4]
for x in a:
    print(x)
```

Indexação

Os elementos em uma lista podem ser acessados de forma individual por meio de **índices**. O índice indica a posição do elemento a ser acessado. Repare que o índice da primeira posição é igual a zero.

```
a = ['x', 'y', 'z', 'w']  
print(a[0]) # x  
print(a[1]) # y  
print(a[2]) # z  
b = a[3]  
print(b)    # w
```

x	y	z	w
0	1	2	3

Indexação

Exemplo: somando os elementos de uma lista com valores inteiros.

```
a = [1, 2, 3, 4]
soma = a[0] + a[1] + a[2] + a[3]
print(soma)
```

Indexação

Exemplo: somando os elementos de uma lista com valores inteiros.

```
# solução com while e índices
a = [2, 4, 6]
soma = 0
i = 0
while i < len(a):
    soma = soma + a[i]
    i = i + 1
print(soma)
```

```
# solução com for e
# índices
a = [2, 4, 6]
soma = 0
for i in
range(len(a)):
    soma = soma + a[i]
print(soma)
```

```
# solução com for
# sem índices
a = [2, 4, 6]
soma = 0
for e in a:
    soma = soma + e
print(soma)
```

Indexação

O índice deve ser um valor inteiro. Consequentemente, variáveis inteiras e valores negativos podem ser utilizados como índices.

```
a = ['x', 'y', 'z', 'w']  
x = 0  
print(a[x])    # x  
print(a[-1])   # w  
print(a[-2])   # z  
print(a[-3])   # y  
print(a[-4])   # x
```

-4	-3	-2	-1
x	y	z	w
0	1	2	3

Indexação

Exemplo: imprimindo os elementos de uma lista em ordem inversa.

```
# solução com while
s = ['x', 'y', 'z', 'w']
n = len(s) * -1
i = -1
while i >= n:
    print(s[i])
    i += -1
```

```
# solução com for e range
s = ['x', 'y', 'z', 'w']
n = len(s) * -1
for i in range(-1, n - 1, -1):
    print(s[i])
```

Indexação

Índices também podem ser usados para alterar os elementos de uma lista.

```
a = [3, 5, 7]
print(a) # imprime [3, 5, 7]
a[0] = 10
a[1] = 6
a[2] = -8
print(a) # imprime [10, 6, -8]
```

Exercícios

1) Qual é a saída (impressão) do programa abaixo?

```
L = [51, 8, 31, 11, 1, 56]
i = 2
print(L[0])
print(L[3])
print(L[i])
print(L[i + 1])
print(L[-1])
print(L[-3])
print(L[-i])
print(L[L[4]])
print(L[2] - L[-4])
```

Indexação

Seja n o tamanho de uma lista, o intervalo de índices válidos é $[-n, n-1]$. Índices com valores fora deste intervalo geram um erro do tipo `IndexError`.

```
a = ['x', 'y', 'z']  
print(a[4])    # gera erro  
print(a[10])   # gera erro  
print(a[-5])   # gera erro
```

Inserção de elementos

Uma lista é dinâmica. Isto significa que novos elementos podem ser adicionados a uma lista. O método `append` insere um novo elemento no fim da lista, enquanto que o método `insert` insere um novo elemento na posição indicada.

```
a = ['A', 'B', 'C']  
a.append('D')  
print(a)           # ['A', 'B', 'C', 'D']  
a.insert(1, '*')  
print(a)           # ['A', '*', 'B', 'C', 'D']
```

Criando uma lista com valor repetido

Podemos usar o operador `*` para criar uma lista com o mesmo valor em todas as suas posições.

```
a = [0] * 5
print(a)      # [0, 0, 0, 0, 0]
b = [None] * 4 # None representa um valor vazio
print(b)      # [None, None, None, None]
```

Copiando uma lista

Uma lista deve ser copiada através do método `copy`. A atribuição entre variáveis faz com que ambas variáveis operem sobre a mesma lista ao invés de criar uma cópia da lista.

```
a = ['x', 'y', 'z']  
b = a  
c = a.copy()  
b[0] = 'W'  
print(a)    # ['W', 'y', 'z']  
print(b)    # ['W', 'y', 'z']  
print(c)    # ['x', 'y', 'z']
```

Instrução in

Podemos usar a instrução `in` para verificar se uma lista contém um valor específico.

```
a = [10, 20, 30]
b = 30
print(5 in a)      # False
print(20 in a)     # True
print(b in a)      # True
```

Exercícios

2) Qual é a saída (impressão) do programa abaixo?

```
ls = [3, 5, 2]
s = 0
for v in ls:
    s = s + v
print(s)
```

Exercícios

3) Qual é a saída (impressão) do programa abaixo?

```
n = 6
ls = [0] * n
for i in range(n):
    ls[i] = i * 2
print(ls)
```

Exercícios

4) Qual é a saída (impressão) do programa abaixo?

```
n = 5
ls = []
for i in range(n):
    ls.append(1)
ls[0] = 5
ls[3] = -3
print(ls)
```

Exercícios

5) Qual é a saída (impressão) do programa abaixo?

```
la = ['A', 'B', 'C']  
lb = [4, 2, 3]  
for i in range(len(la)):  
    print(la[i] * lb[i])
```

Exercícios

6) Escreva uma função que tem uma lista como parâmetro e que retorna a soma dos elementos dessa lista. Escreva pelo menos dois testes da função no programa principal.

Exemplo de lista	Retorno esperado
[2, 5, 3]	10
[1, 0, 9, 1, 5]	16

```
def somar_lista(L):  
    soma = 0  
    for i in L:  
        soma += i  
    return soma  
  
# teste 1  
print( somar_lista([2, 5, 3]) )  
# teste 2  
print( somar_lista([1, 0, 9, 1, 5]) )
```

Exercícios

7) Escreva uma função que tem uma lista como parâmetro e que retorna a média aritmética dos elementos dessa lista. Escreva pelo menos dois testes da função no programa principal.

Exemplo de lista	Retorno esperado
[4, 5, 3]	4.0
[1, 0, 9, 1, 5]	3.2

Exercícios

8) Escreva uma função que tem uma lista como parâmetro e que retorna a soma dos elementos divisíveis por 3. Escreva pelo menos dois testes da função no programa principal.

Exemplo de lista	Retorno esperado
[0, 6, -1, 12, 1]	18.0
[0, 1, -1, 8]	0.0

Exercícios

9) Escreva uma função que tem uma lista como parâmetro e que retorna uma nova lista contendo os elementos da primeira lista que são divisíveis por 5. Escreva pelo menos dois testes da função no programa principal.

Exemplo de lista	Retorno esperado
[1, 1, 10, 15, 9]	[10, 15]
[9, 1, -1, 8]	[]

Exercícios

10) Escreva uma função que tem uma lista como parâmetro e que imprime os elementos pares dessa lista com seus respectivos índices. Escreva pelo menos dois testes da função no programa principal.

Exemplo de lista	Impressão esperada	
[2, 1, -1, 8]	Elemento	Índice
	-----	-----
	2	0
	8	3
[3, 1, -1, 5, 9, 13]	Elemento	Índice
	-----	-----

Exercícios

11) Escreva uma função que tem uma lista **A** como parâmetro e que retorna uma nova lista formada pelos dobros dos elementos de **A**. Escreva pelo menos dois testes da função no programa principal.

Exemplo de lista	Retorno esperado
[2, 1, -1, 8]	[4, 2, -2, 16]
[3, 1, -1, 5, 9, 13]	[6, 2, -2, 10, 18, 26]

Exercícios

12) Escreva uma função que tem duas listas **A** e **B** como parâmetros e que retorna uma nova lista contendo os elementos presentes tanto em **A** como em **B** (interseção). Escreva pelo menos três testes da função no programa principal.

Exemplo de parâmetros	Retorno esperado
[2, 1, -1, 8] [0, 2, 5, 6, 9, 1, 3]	[2, 1]
[3, 1, -1, 5, 9, 13] [0, 1, 5, 4, 5, 6, 5]	[1, 5]
[2, 4, 6, 8, 10] [1, 3, 5]	[]

Referências

MENEZES, Nilo N. C.; **Introdução à Programação com Python**: Algoritmos e lógica de programação para iniciantes. 2ª ed. São Paulo: Novatec, 2014.

FORBELLONE, André; EBERSPACHER, Henri. **Lógica de Programação**: A Construção de Algoritmos e Estruturas de Dados. 3ª ed. Pearson, 2007.